Real-Time Entropy Monitoring and Randomness Assessment in Quantum Random Number Generators

1st Vikram Jhonwal
Department of Computer Science and Engee.
Shri G. S. Institute of Technology and Science, 23 Sir M Visvesvaraya Marg, Indore, 452003 India
Email:vikramjhonwal423@gmail.com

2nd Thomas Andrews Joseph
Quantum Computing Laboratory, Department of Applied Physics and Optoelectronics,
Shri G. S. Institute of Technology and Science, 23 Sir M Visvesvaraya Marg, Indore, 452003 India
jtandrews@sgsits.ac.in, ORCID: 0000-0002-5762-3810

3rd Ashish Singh Bais
Quantum Computing Laboratory, Department of Applied Physics and Optoelectronics,
Shri G. S. Institute of Technology and Science, 23 Sir M Visvesvaraya Marg, Indore, 452003 India
ORCID: 0000-0003-3249-9240

***Abstract*—Cryptographic security, blockchain protocols, and secure communication systems rely fundamentally on the unpredictability of random number generators (RNGs). While classical Pseudo-Random Number Generators (PRNGs) are deterministic and bound to algorithmic vulnerabilities, Quantum Random Number Generators (QRNGs) leverage the inherent non-determinism of quantum mechanics to offer true randomness. However, physical QRNG hardware is susceptible to real-world imperfections, environmental noise, and component degradation, leading to statistical drift and entropy loss. This paper presents a comprehensive framework for real-time entropy monitoring and randomness assessment of QRNGs. We implement a quantum state verification mechanism using Qiskit, execute statistical validations across varying bit samples, model three distinct forms of hardware degradation (bias, noise, and time-dependent drift), and design a consolidated metric termed the Entropy Health Index (EHI). Furthermore, we evaluate post-processing extraction techniques—specifically the Von Neumann extractor and cryptographic hashing—to restore maximal entropy to compromised sequences. Our experimental and mathematical evaluations demonstrate that real-time monitoring can actively detect sub-resolution degradation, preserving the integrity of quantum-driven cryptographic systems.**

***Index Terms*—Quantum Random Number Generator, Entropy Estimation, Shannon Entropy, Min-Entropy, Degradation Modelling, Entropy Health Index, Post-Processing Extractor.**

I. Introduction

The cornerstone of modern cryptographic infrastructures—ranging from secure online banking and one-time pads to advanced blockchain consensus mechanisms—is the absolute unpredictability of numeric keys. If an adversary can anticipate or computationally deduce a random sequence, the underlying cryptographic primitive collapses completely.

Historically, computational systems have relied on Pseudo-Random Number Generators (PRNGs). PRNGs employ deterministic algorithms initialized by a seed value. Although their output sequences satisfy various statistical distributions, they possess a finite period and remain inherently vulnerable to state-compromise attacks. Conversely, Quantum Random Number Generators (QRNGs) exploit the probabilistic nature of quantum state measurements, such as photon polarization, phase fluctuations, or qubit superposition states. According to the foundational postulates of quantum mechanics, the collapse of a wavefunction into a basis state is fundamentally non-deterministic, yielding true randomness.

Despite the theoretical perfection of quantum mechanics, physical deployments of QRNG systems encounter severe engineering bottlenecks. Real-world quantum hardware does not operate in an isolated vacuum. It is continuously subjected to thermal noise, laser intensity fluctuations, semiconductor aging, and electromagnetic interference. These environmental disturbances introduce systemic biases into the bitstream, causing the output to deviate from the ideal uniform distribution (50% zeros and 50% ones). A degraded QRNG introduces predictability, creating backdoors for targeted cryptanalysis.

To mitigate this risk, this paper establishes a systematic architectural framework for an active, real-time telemetry system that continuously assesses, evaluates, models, and corrects the output stream of a QRNG. The key contributions of this work are organized around an 8-step execution methodology: developing a quantum backend generator, evaluating multi-tier mathematical entropy, executing strict NIST-aligned statistical tests, introducing controlled degradation vectors, constructing an algorithmic Entropy Health Index (EHI), assessing post-processing extraction, and presenting a comparative paradigm between classical and quantum randomness.

—

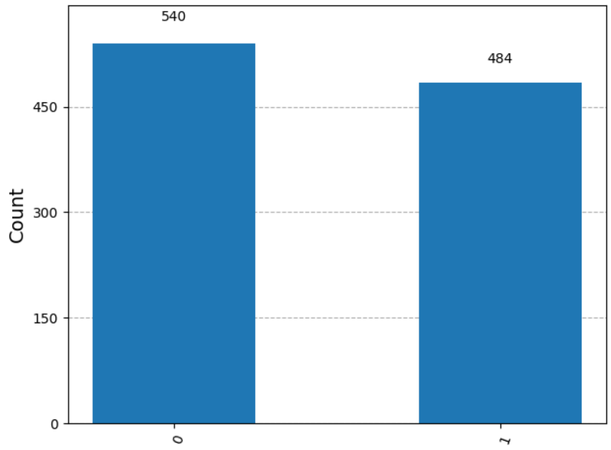

Fig. 1. uniform bit distribution 0 1

II. QRNG Development and Quantum Execution

To construct a foundational QRNG, we model a pure quantum state using a single-qubit subsystem initialized in the ground state $|0\rangle$. True randomness is achieved by driving the qubit into a maximal superposition state utilizing a Hadamard gate (H).

The state transformation is mathematically expressed as:

$$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \tag{1}$$

When a measurement operator in the computational basis (Z-basis) is applied to this state, the wave function collapses to either state $|0\rangle$ or $|1\rangle$ with equal probabilities. The probability $P(\omega)$ for an outcome $\omega \in \{0, 1\}$ is governed by Born's Rule:

$$P(0) = |\langle 0|H|0\rangle|^2 = \frac{1}{2}, \quad P(1) = |\langle 1|H|0\rangle|^2 = \frac{1}{2} \tag{2}$$

Our system implements this pipeline using the IBM Qiskit framework. Algorithm 1 details the loop structures utilized to ingest individual quantum measurement outcomes into a contiguous binary sequence of length N. For larger scale datasets, we execute high-count multi-qubit parallel sampling to prevent execution latencies.

—

Algorithm 1 Qiskit-Based True Random Bit Generation

Require: Required bit length N
Ensure: Binary sequence $B \in \{0, 1\}^N$
1: Initialize $B \leftarrow \emptyset$
2: **while** length(B) $< N$ **do**
3: Create a quantum circuit with 1 Qubit and 1 Classical Bit
4: Apply Hadamard Gate: `circuit.h(0)`
5: Measure Qubit: `circuit.measure(0, 0)`
6: Execute circuit on IBM Quantum backend or local simulator with `shots=1`
7: Extract bit $b \in \{0, 1\}$ from execution result
8: Append b to B
9: **end while**
10: **return** B

III. Mathematical Entropy Estimation

Entropy serves as the definitive metric to quantify the uncertainty and unpredictable information density contained within the generated bit stream. Rather than relying on a single definition, our framework computes three distinct mathematical orders of Renyi entropy to obtain a comprehensive profile of randomness quality.

A. Shannon Entropy

Shannon entropy measures the average uncertainty per symbol in a sequence. For a discrete random variable X outputting values from a binary alphabet $\mathcal{X} = \{0, 1\}$ with probability distribution $P(x)$, the Shannon entropy $H_1(X)$ is defined as:

$$H(X) = -\sum_{x \in \mathcal{X}} P(x) \log_2 P(x) \tag{3}$$

For a perfectly unbiased sequence where $P(0) = P(1) = 0.5$, $H(X) = 1.0$ bit per symbol.

B. Min-Entropy

Min-entropy is the strictest financial and cryptographic metric, focusing entirely on the worst-case scenario. It bounds the probability of the most likely outcome, representing how easily an adversary could guess the sequence in a single attempt. It is formalized as:

$$H_\infty(X) = -\log_2 \left(\max_{x \in \mathcal{X}} P(x) \right) \tag{4}$$

If a hardware defect biases the sequence such that $P(1) = 0.7$, the Shannon entropy might still hover around 0.881, but the Min-entropy drops strictly to $-\log_2(0.7) \approx 0.514$, flagging immediate security vulnerabilities.

C. Collision Entropy

Collision entropy reflects the probability that two independent samples drawn from the same distribution yield identical outcomes. It is defined as:

$$H_2(X) = -\log_2 \sum_{x \in \mathcal{X}} P(x)^2 \tag{5}$$

This metric serves as a robust intermediate indicator between the average-case analysis of Shannon entropy and the worst-case bounds of Min-entropy.

D. Empirical Scale Evaluation

To analyze how sample constraints affect entropy calculation accuracy, we evaluated sequences across sizes $N \in \{100, 500, 1000, 5000, 10000\}$. Small sample sizes ($N = 100$) exhibit statistical fluctuations where sample distributions deviate naturally from the analytical limit due to the Law of Small Numbers. As $N \to 10000$, the experimental value converges asymptotically to the true hardware entropy state.

—

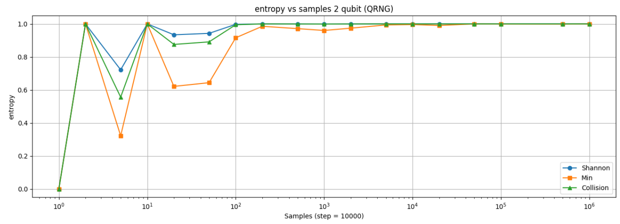

Fig. 2. Entropy Convergence vs. Sample Size ($N = 1, 2, 5, 10, 20, 50, 100, 500, 1000, 2000, 5000, 10000, 20000, 50000, 100000$).

IV. Statistical Randomness Testing

While entropy measures information density, it does not evaluate structural patterns or sequence dependencies. A static deterministic sequence like 01010101... produces a balanced distribution but contains zero cryptographic security. Thus, our architecture integrates real-time statistical tests.

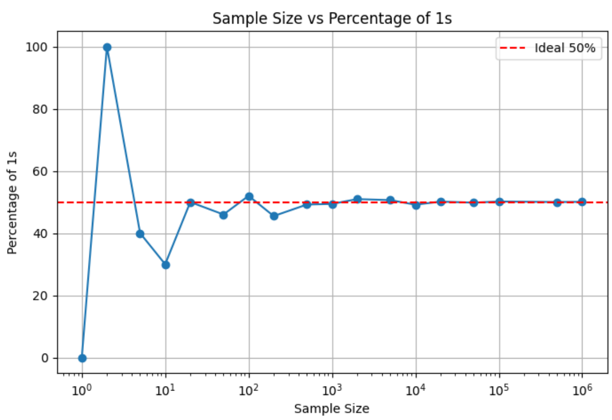

Fig. 3. Sample Size vs. Percentage of Ones (% of 1s converging to 50%).

A. Frequency (Monobit) Test

The primary objective of the Monobit test is to verify whether the proportion of zeros and ones in the stream is approximately equal. Let $x_i \in \{0, 1\}$ be the sequence of bits. We transform each bit into a signed variable $Y_i = 2x_i - 1$, yielding values of -1 or $+1$. The test statistic is calculated as:

$$S_N = \sum_{i=1}^{N} Y_i \implies S_{obs} = \frac{|S_N|}{\sqrt{N}} \tag{6}$$

The corresponding p-value is derived using the complementary error function (erfc):

$$p\text{-value} = \operatorname{erfc}\left(\frac{S_{obs}}{\sqrt{2}}\right) \tag{7}$$

A sequence passes the frequency test if p-value ≥ 0.01.

B. Runs Test

A run is an uninterrupted sequence of identical bits. The Runs test checks if the frequency of transitions between 0 and 1 occurs at an expected random rate. First, the pre-test requires passing the Monobit test. Let π be the proportion of ones: $\pi = \frac{1}{N}\sum x_i$. The total number of runs is denoted by V_N. The expected run count is modeled as:

$$V_N = \sum_{j=1}^{N-1} (x_j \oplus x_{j+1}) + 1 \tag{8}$$

The analytical evaluation calculates:

$$p\text{-value} = \operatorname{erfc}\left(\frac{|V_N - 2N\pi(1-\pi)|}{2\sqrt{2N}\pi(1-\pi)}\right) \tag{9}$$

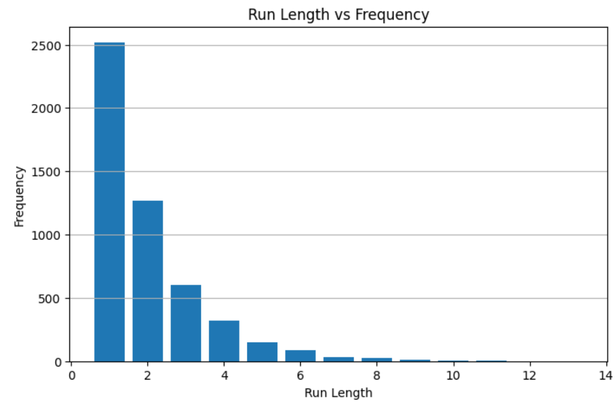

Fig. 4. Run Length vs. Frequency Distribution showing exponential decay in random sequences.

C. Autocorrelation Test

Autocorrelation evaluates the linear dependence between a bit x_i and a shifted version of the bitstream x_{i+d} separated by a fixed lag d. The correlation coefficient $R(d)$ is defined by:

$$R(d) = \frac{1}{N-d}\sum_{i=1}^{N-d} (2x_i - 1)(2x_{i+d} - 1) \tag{10}$$

For completely independent random bits, $R(d)$ should closely fluctuate around 0 across all tested lag indices ($1 \leq d \leq N/10$).

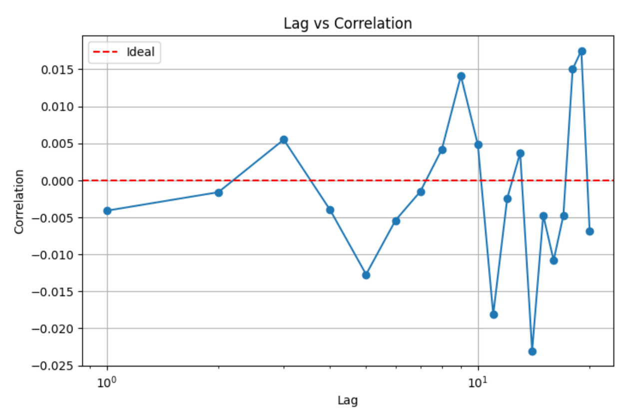

Fig. 5. Autocorrelation Test: Lag vs. Correlation Coefficient ($R(d)$ bounding near 0).

—

V. Degradation Modelling

To evaluate the resiliency and responsiveness of our real-time tracking architecture, we design and inject three controlled physical hardware defect models into an initially ideal quantum bitstream.

A. Case 1: Bias Injection

Hardware bias occurs when the physical detector exhibits a preference for a specific state. We implement an adjustable bias injection parameter $\epsilon \in [0, 0.5]$. The ideal quantum bit b_i is modified via:

$$P(b_i^* = 1) = P(b_i = 1) \cdot (1 - \epsilon) + \epsilon \tag{11}$$

As ϵ scales upwards, the probability of ones transitions toward 100%, inducing an exponential collapse in Shannon, Min, and Collision entropies.

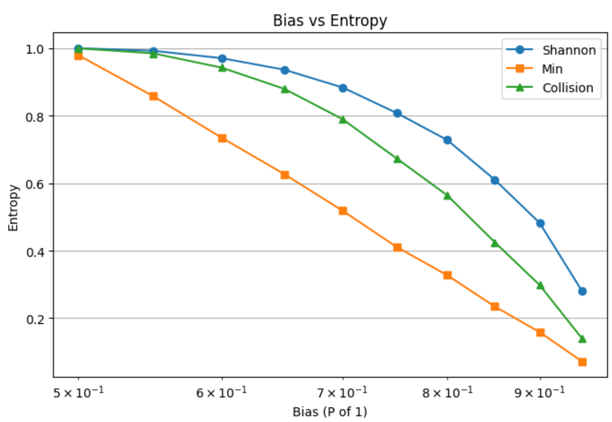

Fig. 6. Case 1: Bias Injection (ϵ) vs. Total Entropy Collapse.

B. Case 2: Noise Injection

Environmental noise models white-noise Gaussian thermal interference affecting high-speed discrimination circuits. We simulate this by implementing a bit-flip probability mechanism governed by noise index $\eta \in [0, 1]$:

$$b_i^* = \begin{cases} b_i & \text{with probability } 1 - \eta \\ 1 - b_i & \text{with probability } \eta \end{cases} \tag{12}$$

Interestingly, if $\eta = 0.5$, the output sequence is driven to maximal entropy (completely chaotic noise), but structural patterns under the runs test break down because physical transitions decouple from the quantum core.

C. Case 3: Time-Dependent Drift Modelling

In deployed systems, laser diode intensities degrade over time, and ambient operating temperatures change throughout day-night cycles. We model this dynamic drift using a time-variant function:

$$\epsilon(t) = \epsilon_0 + \alpha \cdot t + \beta \sin(\omega t) \tag{13}$$

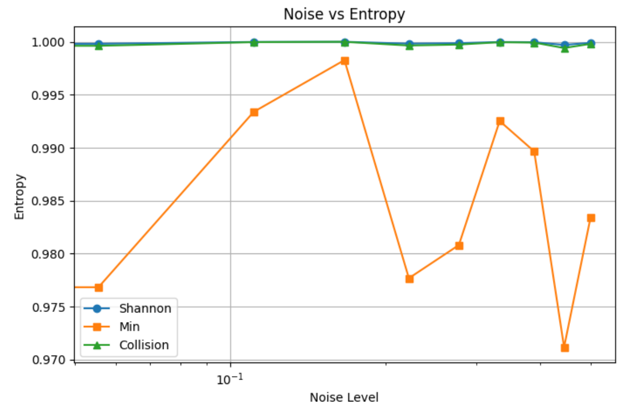

Fig. 7. Case 2: Noise Level (η) vs. Entropy and Pattern Fluctuations.

where α denotes the linear aging vector and $\beta \sin(\omega t)$ simulates cyclical thermal fluctuations. This produces an active, time-varying entropy signature that demands continuous monitoring.

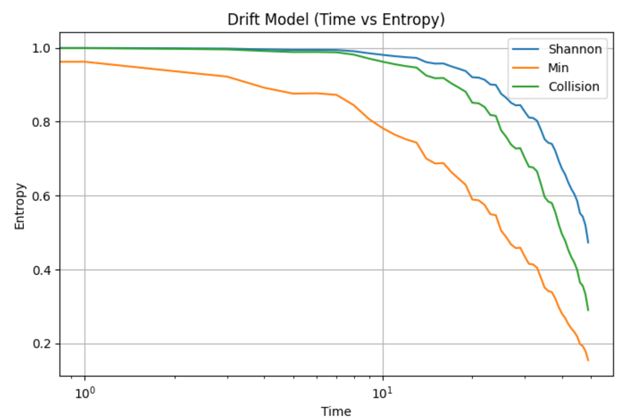

Fig. 8. Case 3: Time-Dependent Drift (t) vs. Entropy under simulated thermal variations.

VI. Entropy Health Index (EHI) & Real-Time Dashboard

Displaying a series of disconnected floating-point metrics like $H_\infty = 0.941$ or $R(2) = 0.082$ is inefficient for critical operations. Our system condenses these parameters into a single, unified metric: the **Entropy Health Index (EHI)**.

A. Threshold Design

The system assigns performance classifications into five distinct operational bands across all monitoring parameters, as structured in Table I.

B. EHI Mathematical Formulation

The EHI is computed using a weighted geometric mean of individual sub-component scores to ensure that a catastrophic failure in any single metric drops the overall score exponentially.

TABLE I
PARAMETER CLASSIFICATION AND THRESHOLD TIERS

Metric	Worst	Warning	Average	Good	Excellent
Shannon (H)	< 0.80	0.80–0.90	0.90–0.95	0.95–0.98	> 0.98
Min (H_∞)	< 0.70	0.70–0.85	0.85–0.92	0.92–0.96	> 0.96
Freq (p-val)	< 0.01	0.01–0.05	0.05–0.20	0.20–0.50	> 0.50
Bias (ϵ)	> 0.20	0.10–0.20	0.05–0.10	0.01–0.05	< 0.01

Let S_E be the entropy composite score, S_T be the statistical test score, and S_D be the degradation score. Each sub-score is normalized dynamically onto a $[0, 100]$ scale:

$$\text{EHI} = (S_E^{w_1} \cdot S_T^{w_2} \cdot S_D^{w_3})^{\frac{1}{w_1+w_2+w_3}} \tag{14}$$

The empirical weights are balanced based on cryptographic vulnerability criteria: Entropy weight $w_1 = 0.50$, Statistical Testing weight $w_2 = 0.30$, and Physical Degradation tracking weight $w_3 = 0.20$.

C. Real-Time Dashboard Architecture

The active monitoring system processes incoming bitstreams in continuous blocks of $N = 1000$ bits every 5 seconds. The dashboard pipeline processes data across five integrated visualization panels:

- **Panel 1: Current Health:** Displays EHI % score with dynamic color notifications (e.g., `EHI = 96%: Status = Excellent`).
- **Panel 2: Entropy Values:** Real-time numeric feed tracking Shannon (0.998), Min (0.987), and Collision (0.992) indices concurrently.
- **Panel 3: Statistical Tests:** Binary indicator grid signaling pass/fail updates for Frequency, Runs, and Autocorrelation metrics.
- **Panel 4: Degradation Indicators:** Live telemetry evaluating component shifts ($\epsilon = 0.8\%$, $\eta = 0.5\%$, drift $= 1.2\%$).
- **Panel 5: Live Graph:** A scrolling time-series line graph illustrating t versus EHI(t) to track performance anomalies.

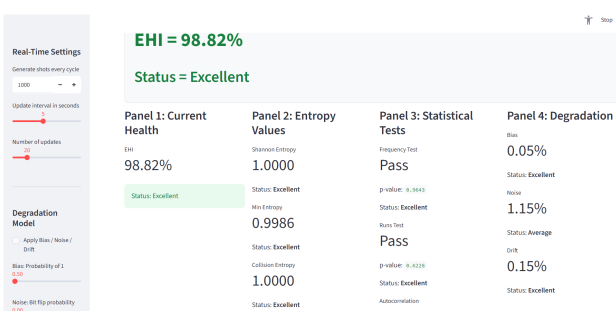

Fig. 9. Real Time Entropy Monitering Dashboard

VII. CRYPTOGRAPHIC RANDOMNESS CORRECTION METHODS

When the real-time dashboard flags an active degradation state, post-processing extraction algorithms are deployed to restore high-entropy compliance.

A. Von Neumann Extractor

The Von Neumann extractor operates on non-overlapping consecutive bit pairs from an asymmetric bitstream with a static bias. The algorithm processes incoming bit pairs according to the rules in Table II.

TABLE II
VON NEUMANN EXTRACTOR MAPPING TABLE

Input Pair	Output Bit	Status
00	No Output	Discarded
11	No Output	Discarded
01	0	Accepted
10	1	Accepted

Let the input bits have a bias such that $P(1) = p$ and $P(0) = q = 1 - p$. The probabilities of the transitions are:

$$P(01) = qp, \quad P(10) = pq \tag{15}$$

Because $qp = pq$, the extracted bits are perfectly balanced (50% probability each), eliminating structural bias. However, this method discards at least 75% of incoming data and fails if the input bits exhibit correlation rather than independent bias.

B. Cryptographic Hash Extractor

To resolve dependent multi-bit structural leaks without significant throughput loss, we deploy a Hash Extractor using the SHA-256 cryptographic hashing standard. The degraded bitstream is partitioned into blocks of size M, where $M > 256$, and compressed:

$$B_{\text{extracted}} = \text{SHA-256}(B_{\text{degraded}}) \tag{16}$$

According to the Avalanche Effect in cryptographic construction, any minor structural shift in the input block creates uncorrelated variations across the entire 256-bit hash space, restoring Min-entropy to near-ideal limits (> 0.999 bits/symbol).

VIII. COMPARATIVE ANALYSIS: PRNG VS. QRNG

To demonstrate the advantage of quantum-driven architectures over algorithmic alternatives, we analyzed sequences from a standard Linear Congruential Generator (LCG) against our Qiskit-driven QRNG under various system constraints.

An LCG updates its internal state via the relation:

$$X_{n+1} = (aX_n + c) \pmod{m} \tag{17}$$

While highly efficient, if an adversary captures 3 to 4 consecutive outputs, the entire matrix can be inverted using simple modular arithmetic. Under our real-time monitoring paradigm, the PRNG tracks high Shannon entropy values ($H \approx 0.999$), masking its predictability. However, when subjected to the

Autocorrelation test across higher lag parameters or evaluated after long-duration operations, structural periodic repeats appear. The QRNG, by contrast, shows no periodic dependencies, confirming its cryptographic superiority.

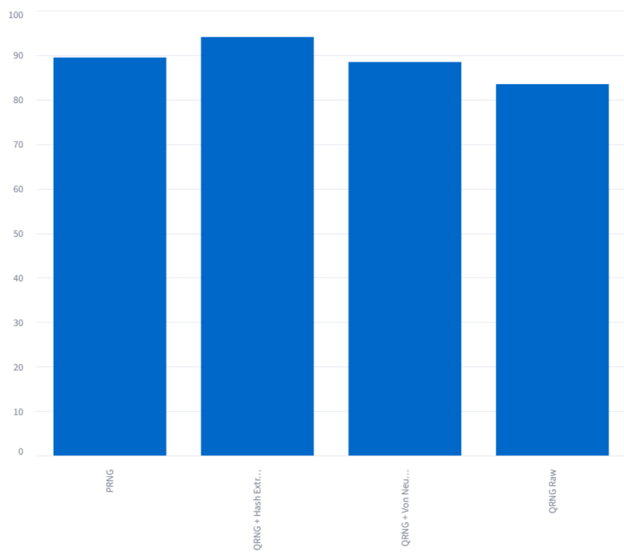

Fig. 10. Real Time Entropy Monitering Dashboard

—

IX. Conclusion and Future Scope

This paper demonstrates an integrated real-time framework to monitor, assess, and correct the entropy output of Quantum Random Number Generators. By implementing multi-tier entropy tracking (Shannon, Min, and Collision entropy) alongside rigorous statistical tests, our architecture can identify subtle hardware degradations like bias, thermal noise, and operational drift.

By mapping these indicators into a unified Entropy Health Index (EHI), our framework provides an actionable health metric for secure systems. When degradation is detected, post-processing extractors like the Von Neumann or cryptographic hash extractors can successfully mitigate bias and restore maximal entropy.

Future research will focus on integrating these entropy assessment algorithms directly onto hardware platforms using Field Programmable Gate Arrays (FPGAs) to support ultra-high throughput line rates. This step will enable low-latency, real-time quantum cryptographic keys for advanced post-quantum security frameworks.

—

References

[1] A. Rukhin et al., “A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications,” NIST Special Publication 800-22 Revision 1a, 2010.

[2] Qiskit Contributors, “Qiskit: An Open-Source SDK for Quantum Computing,” 2024. doi:10.5281/zenodo.2575518.

[3] C. E. Shannon, “A Mathematical Theory of Communication,” Bell System Technical Journal, vol. 27, pp. 379–423, 1948.

[4] Alfréd Rényi, “On measures of entropy and information,” Proceedings of the Fourth Berkeley Symposium on Mathematical Statistics and Probability, 1961.

[5] M. Santha and U. Vazirani, “Generating Quasi-Random Sequences from Semi-Random Sources,” Journal of Computer and System Sciences, 1986.